Network Diagnostics and Path Analysis Tool

Person 2 Report: Diagnostic Tool, SDN Integration, Testing and Evaluation

Prepared by Rakshita Kamarthi (Person 2) · SDN–Socket Programming Mini Project · 31 October 2026

1. Overview

Problem statement. Develop a socket-based network diagnostic application that measures delay, packet loss and path characteristics, and correlates the results with the SDN network topology. The expected features are: UDP diagnostic probes, round-trip time (RTT) measurement, packet-loss detection, path analysis, SDN topology discovery, correlation of application measurements with network paths, and identification of failed paths.

Role of Person 2. The assigned task was to write the script that polls the controller REST API (/stats/switches and /v1.0/topology/links) and formats the final diagnostic report (diagnostic_tool.py), and to assemble the D1 report. This document covers that tool, how it was integrated with the rest of the system, and the testing and performance evaluation carried out with it.

Contribution statement. Rakshita Kamarthi (Person 2) built and ran the diagnostic part of the system and the evidence for it:

- diagnostic_tool.py: polls the controller REST API (switches, links, hosts, port counters), detects failed links and paths, infers the active path, correlates it with the UDP probe results, and prints the diagnostic report.
- Set up the test environment on Ubuntu (WSL): Mininet, Open vSwitch, and Ryu 4.34 in a Python 3.9 environment, including solving the installation problems listed in section 3.
- Ran and verified the full system: REST API checks, normal run, link-failure and recovery tests, the three challenging scenarios, and the static-versus-dynamic performance evaluation.
- Collected the evidence (screenshots 01 to 21, results.csv, results.png, controller logs) and organised the shared GitHub repository (src/, screenshots/, results/, docs/).
- Wrote this report.

Person 1: [FILL: what Person 1 built, for example the UDP server and client.]

Person 3: [FILL: what Person 3 built, for example the Mininet topology and controller.]

AI assistance. The source code of the diagnostic tool, the controller, the topology script and the evaluation script was developed with the assistance of Claude (Anthropic). The author set up the environment, ran, tested and debugged the system, and reviewed the results.

2. System architecture

Component	Where it runs	Purpose
UDP echo server (udp_server.py)	host h2 (10.0.0.2), UDP port 5005	Reflects each probe back to the sender
UDP probe client (udp_client.py)	host h1 (10.0.0.1)	Sends numbered probes, measures RTT and loss, writes /tmp/diag_result.json
Mininet topologies (sdn_topo.py)	Ubuntu (WSL)	Open vSwitch switches, OpenFlow 1.3, remote controller
SDN controller (Ryu 4.34 + reroute_controller.py)	Ubuntu, OpenFlow port 6653	Learns hosts, installs shortest-path flows, reroutes on link events
REST apps (ryu.app.ofctl_rest, ryu.app.rest_topology)	same controller, port 8080	Expose switches, links, hosts, port and flow statistics
Diagnostic tool (diagnostic_tool.py)	normal terminal on Ubuntu	Combines probe results with REST data and prints the report

```
h1 udp_client.py --UDP 5005--> h2 udp_server.py (echo)
      | writes /tmp/diag_result.json (RTT, loss, jitter)
      v
diagnostic_tool.py --REST 8080--> Ryu (ofctl_rest + rest_topology)
                                      | OpenFlow 1.3 (port 6653)
                                      v
                                Open vSwitch switches s1..s5
```

The probes must run inside Mininet, because the hosts live in their own network namespaces and cannot be reached from a normal terminal. The diagnostic tool therefore runs outside Mininet, reads the probe results from the file written by the client, and queries the controller over REST.

3. Environment and setup

Item	Version / detail
Operating system	Ubuntu under WSL (system Python 3.14)
Mininet	2.3.0
Open vSwitch	3.7.1
Controller	Ryu 4.34 in a Python 3.9 virtual environment (created with uv)
eventlet	0.30.2

```
sudo apt install -y mininet openvswitch-switch python3-matplotlib curl
python3 -m venv ~/uvtool && ~/uvtool/bin/pip install uv
~/uvtool/bin/uv venv --seed --python 3.9 ~/ryu39
source ~/ryu39/bin/activate
pip install "setuptools<58" wheel pbr
export PBR_VERSION=4.34
pip install --no-build-isolation --no-deps ryu==4.34
pip install eventlet==0.30.2 msgpack netaddr oslo.config ovs packaging routes
six tinyrpc==1.0.4 webob
ryu-manager --version
```

Problems met during installation and how they were solved

Problem	Cause	Fix
python3.9 not found	The Ubuntu release no longer ships Python 3.9	Installed Python 3.9 with uv into a separate virtual environment
pip install ryu failed (get_script_args)	Ryu's build script is incompatible with modern setuptools on Python 3.14	Used Python 3.9 with setuptools<58
pip installed the ancient Ryu 2.2	Every newer Ryu release reported metadata version 0.0.0	export PBR_VERSION=4.34, then --no-build-isolation --no-deps ryu==4.34
os_ken.cmd missing	The pip package os-ken does not contain the manager command	Dropped OS-Ken and used Ryu
eventlet crashed on Python 3.14	Removed imp module and collections.MutableMapping	Used Python 3.9 with eventlet 0.30.2

Problem	Cause	Fix
curl rejected at the mininet> prompt	Shell commands are not Mininet commands	Ran curl in a separate terminal (or with sh in front)

4. Network topologies

All topologies use OpenFlow 1.3 and a remote controller at 127.0.0.1:6653. Host h1 (10.0.0.1, MAC 00:00:00:00:00:01) is attached to s1; host h2 (10.0.0.2, MAC 00:00:00:00:00:02) is attached to the last switch.

Topology	Switches	Links	Purpose
single	3	s1–s2, s2–s3	One path only: failure test with no alternative
triangle	3	s1–s2, s2–s3, s1–s3	Two paths on three switches
multi	5	s1–s2, s2–s5, s1–s3, s3–s4, s4–s5	Main demonstration topology

In the multi topology (output of the Mininet net command) the ports are: s1-eth1 to s2-eth1, s1-eth2 to s3-eth1, s1-eth3 to h1; s2-eth2 to s5-eth1; s3-eth2 to s4-eth1; s4-eth2 to s5-eth2; s5-eth3 to h2. This gives two candidate paths from h1 to h2:

- Path A: s1 → s2 → s5 (2 switch-to-switch hops, preferred)
- Path B: s1 → s3 → s4 → s5 (3 hops, alternative)

Optional link delay and loss can be added to all switch-to-switch links (--delay 20ms --loss 5).

5. SDN controller used in the tests

The controller (reroute_controller.py) runs on Ryu together with the two REST apps:

```
ryu-manager --observe-links --ofp-tcp-listen-port 6653 --wsapi-port 8080 \
    ryu.app.ofctl_rest ryu.app.rest_topology reroute_controller.py
```

- **Host learning and ARP.** The controller learns each host from packets arriving on host-facing ports and answers ARP requests itself. Unknown ARP and broadcast frames are flooded only to host-facing ports, never to inter-switch ports, so the loops in the topology cannot cause a broadcast storm.

- **Path installation.** For each host pair it computes the shortest path over the links discovered through --observe-links and installs flow entries matching source and destination MAC address in both directions.
- **Dynamic rerouting.** On a link-down event it removes stale flows, recomputes the path over the remaining links and installs the new one. On a link-up event it returns to the preferred shortest path. If no path remains it logs NO_PATH and removes the flows.
- **Static baseline.** Starting the controller with the environment variable REROUTE=0 disables rerouting, so the same topology keeps its original path. This is the baseline used in the performance evaluation.
- **Event log.** Timestamped events (SWITCH_CONNECTED, LINK_UP, LINK_DOWN, PATH_INSTALLED, REROUTE_INSTALLED, NO_PATH) are printed and appended to /tmp/sdn_events.log.

Example from the start-up of the multi topology (screenshot 02): switches s1 to s5 connected, LINK_UP for each inter-switch link in both directions, then PATH_INSTALLED ... path=s1->s2->s5 after the first ping.

6. UDP probing

The client sends --count numbered datagrams (default 10) to the echo server with a send timestamp, waits up to --timeout seconds (default 2.0) for the echo, and records the RTT of each probe. A probe whose echo does not arrive within the timeout is counted as lost. The echoed sequence number is checked so that a late reply to an earlier probe is never counted for the wrong probe.

Results are written to /tmp/diag_result.json (override with --out):

```
{"client": "h1", "server": "10.0.0.2", "sent": 10, "received": 10, "lost": 0,
 "loss_pct": 0.0, "avg_rtt": 0.44, "min_rtt": 0.23, "max_rtt": 1.38,
"jitter": 0.19,
 "probes": [{"seq": 1, "t_sent": 1793.0, "rtt_ms": 1.38, "lost": false},
...]}
```

Jitter is the mean absolute difference between consecutive RTTs of the successful probes.

7. The diagnostic tool (diagnostic_tool.py)

7.1 What it does

1. **Discovers the SDN topology** from the controller REST API: switches (/stats/switches), links (/v1.0/topology/links) and hosts (/v1.0/topology/hosts). DPIDs and port numbers arrive as hexadecimal strings in the topology API and as integers in the statistics API; the tool converts both.
2. **Detects failed links.** The first topology seen is saved as a known-good baseline (--baseline-file). Links in the baseline that are missing from the live topology are reported as failed.
3. **Analyses paths.** It enumerates every simple path between the source and destination switches over the baseline graph and marks a path FAILED if it uses a failed link.
4. **Reads the probe results** written by the UDP client (--from-file, a path or an http URL). Several key spellings and a per-probe list are accepted, and a warning is printed if the file is more than ten minutes old.
5. **Infers the active path.** Transmit counters of every switch port are read from /stats/port/<dpid> before and after the probes (the tool pauses so the probes can be run in Mininet). The candidate path whose egress ports all gained the most packets is reported as the path the probes took.
6. **Correlates application and network.** Loss above the threshold (default 20%) is marked DEGRADED. If a link is missing, the loss is attributed to it; if all links are up, congestion or a bad flow rule is suspected. A reroute suggestion is printed when a live alternative path exists.
7. **Compares with the previous run** (diagnostic_history.json) and can write the report as JSON (--json-out).

7.2 Usage

```
python3 diagnostic_tool.py --ryu 127.0.0.1:8080 --from-file
/tmp/diag_result.json \
    --src-dpid 1 --dst-dpid 5 --baseline-file base_multi.json --reset-
baseline --label normal
```

Option	Meaning
--ryu host:port	Controller REST address
--from-file	Probe results written by the client in h1 (path or URL); without it the tool sends its own probes

Option	Meaning
--src-dpid, --dst-dpid	Ingress and egress switches (found from the host table if omitted)
--baseline-file	Known-good topology snapshot; use one file per topology
--reset-baseline	Overwrite the baseline (use only on a healthy network)
--label	Name of the run, stored in the history
--wait N	Seconds between the two counter snapshots instead of waiting for Enter
--loss-threshold	Loss percentage above which the status is DEGRADED
--json-out	Also write the report as JSON

7.3 Report layout

The report has six sections: topology (from Ryu), path analysis, probe results, measurements, correlation (active path, failed links, failed paths, suggested reroute), and comparison with the previous run.

8. Testing and results

8.1 REST API verification

With Mininet running and pingall completed, the three REST queries returned: [1, 2, 3, 4, 5] for /stats/switches; ten link entries (each of the five links in both directions) for /v1.0/topology/links; and two hosts for /v1.0/topology/hosts, h1 (10.0.0.1) on s1 port 3 and h2 (10.0.0.2) on s5 port 3. pingall reported 0% dropped (2/2 received). Evidence: screenshots 04 to 08.

8.2 Normal run (healthy network)

The UDP client sent 10 probes to 10.0.0.2 and all 10 were answered (0% loss). The diagnostic tool then produced the following report (screenshot 10):

Item	Result
Topology	Switches s1 to s5; links s1–s2, s1–s3, s2–s5, s3–s4, s4–s5
Candidate paths	s1 → s2 → s5 (2 hops, UP); s1 → s3 → s4 → s5 (3 hops, UP)
Sent / received / lost	10 / 10 / 0
RTT min / avg / max	0.23 / 0.44 / 1.38 ms
Jitter	0.19 ms
Status	HEALTHY
Active path (from port counters)	s1 → s2 → s5
Failed links / failed paths	none / none

The port-counter inference agreed with the controller log, which had installed the path s1 → s2 → s5.

8.3 Failure test

Procedure. (1) Normal run with --reset-baseline to save the baseline. (2) In Mininet, link s1 s2 down. (3) Check the controller log for LINK_DOWN and REROUTE_INSTALLED. (4) Confirm traffic still flows (h1 ping -c 3 10.0.0.2). (5) Run the tool again without --reset-baseline.

Expected behaviour. The controller detects the failed link, removes the stale flows and installs s1 → s3 → s4 → s5. The tool then reports the s1–s2 link as failed, the path s1 → s2 → s5 as FAILED, an active path of s1 → s3 → s4 → s5, and a slightly higher RTT than in the normal run because the path is longer.

Observed. The controller log (sdn_events_failure_test.log) shows LINK_DOWN s1->s2 detected at 10:47:36.978, followed 1 ms later by REROUTE_INSTALLED for h1 → h2 with path s1->s3->s4->s5 (elapsed_ms=0.9). The reverse direction was rerouted along s5->s4->s3->s1 (elapsed_ms=0.7). The failure and recovery were repeated three times (10:47, 11:04 and 11:06); in each case the reroute was installed within 0.6 to 1.2 ms of the controller detecting the failure.

The diagnostic tool then reported s1<->s2 as the failed link and the path s1 → s2 → s5 as FAILED, with s1 → s3 → s4 → s5 still UP. The active path inferred from the port counters was s1 → s3 → s4 → s5, and the suggested reroute was the same path. The 10 probes showed

0% loss (10 of 10 received), RTT min / avg / max of 0.26 / 0.51 / 1.34 ms and jitter 0.22 ms, so the status was HEALTHY. Compared with the previous stored run (average RTT 0.42 ms), the average RTT rose by 0.09 ms, which is consistent with the longer path.

A separate 40-probe run (interval 0.2 s, timeout 0.5 s) across the failure also lost no probes (0% loss, average RTT 0.43 ms, jitter 0.12 ms), so the reroute completed faster than the probe interval and no probe timed out. The elapsed_ms value in the controller log is its processing time after it detects the failure; it is not the failover time seen by the probes (see section 9).

Evidence: screenshots 11 to 15 (failure and recovery).

8.4 Recovery

After link s1 s2 up the controller logged LINK_UP s1->s2 at 10:53:49.739 and REROUTE_INSTALLED with path s1->s2->s5 (reason=link_up) 1 ms later. The reverse direction was restored along s5->s2->s1 at 10:53:49.741. A ping from h1 to h2 then succeeded (3 of 3 received, 0% loss). The tool's next report (label "recovered") showed both paths UP, FAILED LINKS: none, FAILED PATHS: none, active path s1 → s2 → s5, 0% loss and average RTT 0.52 ms. Evidence: screenshots 16 and 17.

8.5 Challenging scenarios

Scenario	How it was produced	Expected response	Observed
Two links down	link s1 s2 down and link s3 s4 down	Controller logs NO_PATH; h1 cannot reach h2; connectivity returns after the links are restored	After link s3 s4 went down the controller logged NO_PATH for both directions (11:34:51) and removed the flows. h1 ping failed (3 sent, 0 received, 100% loss) and all 10 UDP probes were lost. The tool reported both paths as FAILED, failed links s1<->s2 and s3<->s4, loss 100% (DEGRADED) and no active path. After both links were restored, the first ping was lost (2 of 3 received) and the next 5 succeeded with 0% loss, so connectivity returned once new traffic reached the controller.
No alternative path	single topology, one link down	No reroute possible; the tool reports the failed link and path with no alternative	LINK_DOWN s1->s2 was logged at 11:52:27.492, followed within 3 ms by NO_PATH for both directions (flows removed). Ping failed (100% loss) and all 10 probes were lost. The tool reported s1<->s2 as the failed link, the only path s1 → s2 → s3 as FAILED, status DEGRADED and no active path. After link s1 s2 up, ping succeeded again (5 of 5, 0% loss).

Scenario	How it was produced	Expected response	Observed
Injected delay and loss	--delay 20ms --loss 5 on the inter-switch links	RTT rises and loss appears in the probe results	The UDP probes showed an average RTT of 81.25 ms (min / max 80.84 / 82.25 ms) with 25.0% loss (10 of 40 probes); an earlier 40-probe run showed 81.27 ms and 17.5% loss. This compares with 0.44 ms and 0% in the normal run. The tool reported all links up and no failed paths, and attributed the loss to congestion, flow rules or netem loss on the active path s1 → s2 → s5. About 80 ms is expected, because a probe crosses two delayed links in each direction (4 × 20 ms). The controller logged no link failure, because delay and loss are not link-down events; only the probe measurements revealed them.

Evidence: screenshots 18 to 20.

After the NO_PATH events the controller has no path to recompute when the links return. The route is installed again by the next packet that reaches the controller (logged as PATH_INSTALLED), which is why the first ping after restoring the links in the two-links-down scenario was lost.

9. Performance evaluation

Method. evaluate.py builds the topology, runs the UDP client in h1 (probe every 0.2 s, timeout 0.3 s), brings the link s1–s2 down 4 s after the start and back up at 14 s, and analyses the probe log. It was run for 5 trials with the controller in dynamic mode and 5 trials with the controller in static mode (REROUTE=0).

Metrics. Packet loss (%), mean RTT before the failure, mean RTT after the failure, failover time (from link-down to the first successful probe while the link is still down) and whether traffic recovered before the link was restored.

Mode	Trials	Avg loss (%)	Avg RTT before (ms)	Avg RTT after (ms)	Avg failover (s)	Recovered before link-up
Dynamic (rerouting)	5	0.0	0.58	0.58	0.08	5 of 5
Static (no rerouting)	5	30.3	0.63	none	none	0 of 5

The raw per-trial data is in results.csv and the chart in results.png (screenshots 21_a to 21_g). In dynamic mode traffic recovered before the link was restored in all 5 trials, with no packet loss, an average failover time of 0.08 s (range 0.07 to 0.09 s) and an average RTT of 0.58 ms after the failure. In static mode traffic did not recover in any of the 5 trials and 20 of 66 probes (30.3%) were lost in every trial, because the installed flows kept pointing at the failed link. Rerouting therefore turned an outage that lasted until the link was repaired into a failover of under 0.1 s.

In the chart, the static bars are empty in the failover-time and RTT-after panels because static mode never recovered, so there is no value to plot. The failover time has a resolution of one probe interval (0.2 s), and the first probe after the failure may be lost while the controller reacts.

10. Requirement traceability

Expected feature	Where implemented	Evidence
UDP diagnostic probes	udp_client.py, udp_server.py	Screenshots 09, 10
Round-trip time	Per-probe RTT, min / avg / max, jitter	Section 8.2
Packet loss detection	Timeout per probe; loss percentage	Sections 6, 8.5
Path analysis	Enumeration of all s1-to-s5 paths; active-path inference from port counters	Section 8.2
SDN topology discovery	Ryu --observe-links, REST topology API read by the tool	Section 8.1
Correlation of measurements with paths	Diagnostic report section 5	Sections 8.2, 8.3

Expected feature	Where implemented	Evidence
Identification of failed paths	Baseline comparison, FAILED paths in the report	Section 8.3
Dynamic SDN behaviour	Rerouting on link events in the controller	Section 8.3
Performance comparison with a static baseline	evaluate.py, REROUTE=0 mode	Section 9

11. Limitations and future work

- The active path is inferred from port counters, so the probes must run while the tool waits between its two snapshots, and other traffic on the same ports can distort the result.
- Flow entries match source and destination MAC addresses and the controller was built for a two-host demonstration; many hosts would need a more general design.
- The baseline topology must be saved once per topology and only while the network is healthy.
- Probes run inside Mininet and the results are passed to the tool through a file, so the tool and Mininet must run on the same machine (or the file must be served over HTTP).
- Failover time is measured with the resolution of the probe interval.
- After a NO_PATH event the route is only reinstalled when new traffic reaches the controller, so recovery after a total outage depends on fresh packets.
- Ryu 4.34 needs Python 3.9 and eventlet 0.30.2, which complicates installation on current Ubuntu releases.
- Possible extensions: continuous monitoring mode, per-flow statistics, automatic selection of the least-loaded path, and a graphical view of the topology.

12. How to reproduce

1. Start the controller (section 5) in one terminal with the Python 3.9 environment active.
2. Start Mininet in a second terminal: sudo python3 sdn_topo.py --topo multi, wait 10 seconds, then run pingall.
3. Run the diagnostic tool in a third terminal (section 7.2). When it pauses, run h1 python3 udp_client.py 10.0.0.2 --count 10 in Mininet and press Enter.
4. For the failure test, run link s1 s2 down in Mininet and run the tool again without --reset-baseline.

5. For the performance evaluation, run sudo python3 evaluate.py --topo multi --mode dynamic --trials 5, restart the controller with REROUTE=0, then run the same command with --mode static.

13. Evidence index

Screenshots	Content
01a to 01d	Controller start-up, apps loaded, REST server listening
02	Switches connected, links discovered, path installed
03	Mininet start-up
04, 05	net, links and pingall (0% dropped)
06, 07, 08	REST outputs: switches, links, hosts
09	UDP client run: 10 probes, 0% loss
10	Diagnostic report, normal run
11, 12	Link s1–s2 down and the controller reroute
13	Ping after the failure (traffic still flows)
14	Diagnostic report, failure run
15	40-probe client run across the failure
16, 17	Link restored, controller reroute back, and the recovery report
18	Challenging scenario: two links down
19	Challenging scenario: no alternative path
20	Challenging scenario: injected delay and loss
21_a to 21_g	Performance evaluation: dynamic and static runs, results.csv, results.png